EXP 2 - Data Handling technique with Programming exercise¶

Dataset: Sub_Division_IMD_2017.csv (IMD subdivision-wise monthly rainfall, India) **Use case:** KJS-AGR-01 (AI Irrigation Advisory) - same dataset used in EXP1 **Roll No:** 16014324020

In [1]:

```
import numpy as np
import pandas as pd

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 150)
```

1. Reading the dataset (NumPy + Pandas)¶

In [2]:

```
df = pd.read_csv("Sub_Division_IMD_2017.csv")
print("Shape:", df.shape)

# quick numpy usage - convert one numeric column to a numpy array
annual_arr = np.array(df['ANNUAL'])
print("ANNUAL as numpy array, dtype:", annual_arr.dtype, "first 5:", annual_arr[:5])
```

```
Shape: (4188, 19)
ANNUAL as numpy array, dtype: float64 first 5: [3373.2 3520.7 2957.4 3079.6 2566.7]
```

2/3. Detailed data analysis - info(), describe(), shape, columns¶

In [3]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4188 entries, 0 to 4187
Data columns (total 19 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   SUBDIVISION  4188 non-null   str
 1   YEAR         4188 non-null   int64
 2   JAN          4184 non-null   float64
 3   FEB          4185 non-null   float64
 4   MAR          4182 non-null   float64
 5   APR          4184 non-null   float64
 6   MAY          4185 non-null   float64
 7   JUN          4183 non-null   float64
 8   JUL          4181 non-null   float64
 9   AUG          4184 non-null   float64
 10  SEP          4182 non-null   float64
```

```
 11  OCT          4181 non-null   float64
 12  NOV          4177 non-null   float64
 13  DEC          4178 non-null   float64
 14  ANNUAL       4162 non-null   float64
 15  JF           4182 non-null   float64
 16  MAM          4179 non-null   float64
 17  JJAS         4178 non-null   float64
 18  OND          4175 non-null   float64
dtypes: float64(17), int64(1), str(1)
memory usage: 621.8 KB
```

In [4]:

```
df.describe()
```

Out[4]:

	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL
count	4188.000000	4184.000000	4185.000000	4182.000000	4184.000000	4185.000000	4183.000000	4181.000
mean	1959.220630	18.938337	21.599427	27.412912	43.071845	85.660860	230.117475	347.0200
std	33.717097	33.793128	35.731734	46.927963	68.150985	122.695137	234.292087	268.8507
min	1901.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.400000	0.000000
25%	1930.000000	0.600000	0.500000	1.000000	3.000000	8.600000	70.800000	175.6000
50%	1959.000000	5.950000	6.500000	7.900000	15.450000	36.900000	138.800000	285.0000
75%	1988.000000	22.000000	26.600000	31.375000	49.650000	97.800000	305.000000	418.5000
max	2017.000000	583.700000	403.500000	605.600000	595.100000	1168.600000	1609.900000	2362.800

In [5]:

```
print('Shape:', df.shape)
print('Columns:', list(df.columns))
```

```
Shape: (4188, 19)
Columns: ['SUBDIVISION', 'YEAR', 'JAN', 'FEB', 'MAR', 'APR', 'MAY', 'JUN', 'JUL',
'AUG', 'SEP', 'OCT', 'NOV', 'DEC', 'ANNUAL', 'JF', 'MAM', 'JJAS', 'OND']
```

4. Null values - check + count¶

In [6]:

```
df.isnull().sum()
```

Out[6]:

```
SUBDIVISION     0
YEAR            0
JAN             4
FEB             3
MAR             6
APR             4
MAY             3
JUN             5
JUL             7
AUG             4
SEP             6
OCT             7
NOV            11
DEC            10
ANNUAL         26
JF              6
MAM             9
JJAS           10
OND            13
dtype: int64
```

In [7]:

```
print('Total nulls in dataset:', df.isnull().sum().sum())
```

```
Total nulls in dataset: 134
```

5. Handling missing values (imputation, using inplace)¶

Filling with **subdivision-wise mean** rather than a single global mean, since rainfall varies hugely by region.

In [8]:

```
month_cols = ['JAN','FEB','MAR','APR','MAY','JUN','JUL','AUG','SEP','OCT','NOV','DEC']
season_cols = ['ANNUAL','JF','MAM','JJAS','OND']

# note: df[col].fillna(val, inplace=True) does NOT work anymore in newer pandas
# (copy-on-write breaks chained assignment) - reassigning df[col] is the fix.
for col in month_cols + season_cols:
    df[col] = df[col].fillna(df.groupby('SUBDIVISION')[col].transform('mean'))

# any subdivision still fully NaN for a column -> fallback to global mean, true inplace
on whole df
df.fillna(df[month_cols + season_cols].mean(numeric_only=True), inplace=True)

print("Nulls remaining after imputation:", df.isnull().sum().sum())
```

```
Nulls remaining after imputation: 0
```

6. Sorting + top 5/8 records (conditional filtering)¶

In [9]:

```
print("Top 5 rows by ANNUAL rainfall:")
df.sort_values(by='ANNUAL', ascending=False).head(5)[['SUBDIVISION','YEAR','ANNUAL']]
```

```
Top 5 rows by ANNUAL rainfall:
```

Out[9]:

	SUBDIVISION	YEAR	ANNUAL
144	Arunachal Pradesh	1948	6331.1
134	Arunachal Pradesh	1938	6129.0
117	Arunachal Pradesh	1921	5691.4
3664	Coastal Karnataka	1961	5553.9
114	Arunachal Pradesh	1918	5486.3

In [10]:

```
print("Top 8 rows by JJAS (monsoon) rainfall, years after 2000:")
df[df['YEAR'] > 2000].sort_values(by='JJAS', ascending=False).head(8)
[['SUBDIVISION','YEAR','JJAS']]
```

```
Top 8 rows by JJAS (monsoon) rainfall, years after 2000:
```

Out[10]:

	SUBDIVISION	YEAR	JJAS
3716	Coastal Karnataka	2013	3727.6
2661	Konkan & Goa	2011	3697.3
3710	Coastal Karnataka	2007	3567.2
2666	Konkan & Goa	2016	3551.0
2655	Konkan & Goa	2005	3522.2
3714	Coastal Karnataka	2011	3505.4
2660	Konkan & Goa	2010	3467.4
2657	Konkan & Goa	2007	3369.9

7. Frequency listing of a column (2 cases)¶

In [11]:

```
print("Case 1 - record count per SUBDIVISION:")
df['SUBDIVISION'].value_counts()
```

```
Case 1 - record count per SUBDIVISION:
```

Out[11]:

```
SUBDIVISION
Assam & Meghalaya                   117
Naga Mani Mizo Tripura              117
Sub Himalayan West Bengal & Sikkim  117
Gangetic West Bengal                117
Orissa                              117
Jharkhand                           117
Bihar                               117
East Uttar Pradesh                  117
West Uttar Pradesh                  117
Uttarakhand                         117
Haryana Delhi & Chandigarh          117
Punjab                              117
Himachal Pradesh                    117
Jammu & Kashmir                     117
West Rajasthan                      117
East Rajasthan                      117
West Madhya Pradesh                 117
East Madhya Pradesh                 117
Gujarat Region                      117
Saurashtra & Kutch                  117
Konkan & Goa                        117
Madhya Maharashtra                  117
Matathwada                          117
Vidarbha                            117
Chhattisgarh                        117
Coastal Andhra Pradesh              117
Telangana                           117
Rayalseema                          117
Tamil Nadu                          117
Coastal Karnataka                   117
North Interior Karnataka            117
South Interior Karnataka            117
Kerala                              117
Lakshadweep                         116
Andaman & Nicobar Islands           112
Arunachal Pradesh                    99
Name: count, dtype: int64
```

In [12]:

```
print("Case 2 - record count per decade:")
df['DECADE'] = (df['YEAR'] // 10) * 10
df['DECADE'].value_counts().sort_index()
```

```
Case 2 - record count per decade:
```

Out[12]:

```
DECADE
1900    314
1910    353
1920    360
1930    360
1940    356
1950    357
1960    360
1970    360
1980    360
1990    360
2000    360
2010    288
Name: count, dtype: int64
```

8. Implicit (iloc) vs explicit (loc) indexing¶

In [13]:

```
print("Explicit indexing with loc - rows 0-4, columns SUBDIVISION to MAR:")
df.loc[0:4, 'SUBDIVISION':'MAR']
```

```
Explicit indexing with loc - rows 0-4, columns SUBDIVISION to MAR:
```

Out[13]:

	SUBDIVISION	YEAR	JAN	FEB	MAR
0	Andaman & Nicobar Islands	1901	49.2	87.1	29.2
1	Andaman & Nicobar Islands	1902	0.0	159.8	12.2
2	Andaman & Nicobar Islands	1903	12.7	144.0	0.0
3	Andaman & Nicobar Islands	1904	9.4	14.7	0.0
4	Andaman & Nicobar Islands	1905	1.3	0.0	3.3

In [14]:

```
print("Implicit indexing with iloc - first 5 rows, first 4 columns:")
df.iloc[0:5, 0:4]
```

```
Implicit indexing with iloc - first 5 rows, first 4 columns:
```

Out[14]:

	SUBDIVISION	YEAR	JAN	FEB
0	Andaman & Nicobar Islands	1901	49.2	87.1
1	Andaman & Nicobar Islands	1902	0.0	159.8
2	Andaman & Nicobar Islands	1903	12.7	144.0
3	Andaman & Nicobar Islands	1904	9.4	14.7
4	Andaman & Nicobar Islands	1905	1.3	0.0

In [15]:

```
print("Columns sorted alphabetically:")
df.sort_index(axis=1).head(2)
```

```
Columns sorted alphabetically:
```

Out[15]:

	ANNUAL	APR	AUG	DEC	DECADE	FEB	JAN	JF	JJAS	JUL	JUN	MAM	MAR	MAY	NOV	C
0	3373.2	2.3	481.1	33.6	1900	87.1	49.2	136.3	1696.3	365.1	517.5	560.3	29.2	528.8	558.2	38
1	3520.7	0.0	753.7	160.5	1900	159.8	0.0	159.8	2185.9	228.9	537.1	458.3	12.2	446.1	359.0	19

9. Accessing rows on a condition, few columns (3 cases, compound conditions)¶

In [16]:

```
print("Case 1 - Konkan & Goa, years after 2010:")
c1 = df[(df['SUBDIVISION'] == 'Konkan & Goa') & (df['YEAR'] > 2010)]
c1[['YEAR','ANNUAL']]
```

```
Case 1 - Konkan & Goa, years after 2010:
```

Out[16]:

	YEAR	ANNUAL
2661	2011	3825.2
2662	2012	3018.4

	YEAR	ANNUAL
2663	2013	3524.0
2664	2014	2757.5
2665	2015	2082.0
2666	2016	3663.9
2667	2017	3443.4

In [17]:

```
print("Case 2 - ANNUAL > 3000mm AND JJAS > 2000mm:")
c2 = df[(df['ANNUAL'] > 3000) & (df['JJAS'] > 2000)]
c2[['SUBDIVISION','YEAR','ANNUAL','JJAS']].head(8)
```

```
Case 2 - ANNUAL > 3000mm AND JJAS > 2000mm:
```

Out[17]:

	SUBDIVISION	YEAR	ANNUAL	JJAS
1	Andaman & Nicobar Islands	1902	3520.7	2185.9
7	Andaman & Nicobar Islands	1908	3576.4	2303.7
32	Andaman & Nicobar Islands	1934	3722.8	2717.9
43	Andaman & Nicobar Islands	1949	3489.6	2433.2
48	Andaman & Nicobar Islands	1954	3449.0	2419.4
53	Andaman & Nicobar Islands	1959	3343.3	2271.7
88	Andaman & Nicobar Islands	1994	3262.8	2191.5
105	Andaman & Nicobar Islands	2011	3828.0	2165.0

In [18]:

```
print("Case 3 - Coastal Karnataka OR Kerala, ANNUAL < 2000mm (low-rainfall year):")
c3 = df[((df['SUBDIVISION'] == 'Coastal Karnataka') | (df['SUBDIVISION'] == 'Kerala'))
& (df['ANNUAL'] < 2000)]
c3[['SUBDIVISION','YEAR','ANNUAL']]
```

```
Case 3 - Coastal Karnataka OR Kerala, ANNUAL < 2000mm (low-rainfall year):
```

Out[18]:

	SUBDIVISION	YEAR	ANNUAL
4070	Kerala	2016	1870.9

10. Min / Max analysis¶

In [19]:

```
print("Max ANNUAL rainfall record:")
print(df.loc[df['ANNUAL'].idxmax(), ['SUBDIVISION','YEAR','ANNUAL']])
print("\nMin ANNUAL rainfall record:")
print(df.loc[df['ANNUAL'].idxmin(), ['SUBDIVISION','YEAR','ANNUAL']])
```

```
Max ANNUAL rainfall record:
SUBDIVISION    Arunachal Pradesh
YEAR                        1948
ANNUAL                    6331.1
Name: 144, dtype: object

Min ANNUAL rainfall record:
SUBDIVISION    West Rajasthan
YEAR                     1918
ANNUAL                   62.3
Name: 1866, dtype: object
```

In [20]:

```
print("Using numpy directly:")
print("max:", np.max(df['ANNUAL'].values), " min:", np.min(df['ANNUAL'].values))
```

```
Using numpy directly:
max: 6331.1  min: 62.3
```

11. Groupby on one / more columns (2 cases)¶

In [21]:

```
print("Case 1 - groupby SUBDIVISION, mean ANNUAL rainfall:")
g1 = df.groupby('SUBDIVISION')['ANNUAL'].mean().sort_values(ascending=False)
g1.head(10)
```

```
Case 1 - groupby SUBDIVISION, mean ANNUAL rainfall:
```

Out[21]:

```
SUBDIVISION
Arunachal Pradesh                  3403.959140
Coastal Karnataka                  3397.637931
Konkan & Goa                       2987.531624
Andaman & Nicobar Islands          2926.373585
```

```
Kerala                                2914.247009
Sub Himalayan West Bengal & Sikkim    2750.552991
Assam & Meghalaya                     2579.133333
Naga Mani Mizo Tripura                2432.717949
Lakshadweep                           1587.294286
Gangetic West Bengal                  1490.612821
Name: ANNUAL, dtype: float64
```

In [22]:

```
print("Case 2 - groupby [SUBDIVISION, DECADE], mean JJAS rainfall:")
g2 = df.groupby(['SUBDIVISION','DECADE'])['JJAS'].mean()
g2.head(15)
```

```
Case 2 - groupby [SUBDIVISION, DECADE], mean JJAS rainfall:
```

Out[22]:

```
SUBDIVISION                DECADE
Andaman & Nicobar Islands  1900      1806.912500
                           1910      1623.320000
                           1920      1620.270000
                           1930      1914.850826
                           1940      1834.802752
                           1950      1826.550000
                           1960      1707.680000
                           1970      1596.100000
                           1980      1630.290000
                           1990      1634.370000
                           2000      1530.210000
                           2010      1814.350000
Arunachal Pradesh          1910      3010.078608
                           1920      3100.860000
                           1930      3087.760000
Name: JJAS, dtype: float64
```

12. New column derived from existing columns¶

In [23]:

```
# irrigation-advisory relevant flag: what share of annual rainfall is from the monsoon?
df['MONSOON_SHARE_PCT'] = (df['JJAS'] / df['ANNUAL']) * 100
df[['SUBDIVISION','YEAR','ANNUAL','JJAS','MONSOON_SHARE_PCT']].head()
```

Out[23]:

	SUBDIVISION	YEAR	ANNUAL	JJAS	MONSOON_SHARE_PCT
0	Andaman & Nicobar Islands	1901	3373.2	1696.3	50.287561
1	Andaman & Nicobar Islands	1902	3520.7	2185.9	62.087085
2	Andaman & Nicobar Islands	1903	2957.4	1874.0	63.366471

	SUBDIVISION	YEAR	ANNUAL	JJAS	MONSOON_SHARE_PCT
3	Andaman & Nicobar Islands	1904	3079.6	1977.6	64.216132
4	Andaman & Nicobar Islands	1905	2566.7	1624.9	63.306970

13. Aggregate functions with groupby (2 cases)¶

In [24]:

```
print("Case 1 - agg() with multiple stats on ANNUAL, per SUBDIVISION:")
agg1 = df.groupby('SUBDIVISION')['ANNUAL'].agg(['mean','min','max','std'])
agg1.head(10)
```

```
Case 1 - agg() with multiple stats on ANNUAL, per SUBDIVISION:
```

Out[24]:

	mean	min	max	std
SUBDIVISION				
Andaman & Nicobar Islands	2926.373585	1849.4	3938.2	395.096714
Arunachal Pradesh	3403.959140	1668.5	6331.1	1038.487876
Assam & Meghalaya	2579.133333	1743.4	3403.5	309.122231
Bihar	1196.563248	629.2	1660.4	193.153686
Chhattisgarh	1369.136752	904.6	1974.0	210.174525
Coastal Andhra Pradesh	1050.304274	703.2	1712.4	189.774185
Coastal Karnataka	3397.637931	2510.9	5553.9	483.734096
East Madhya Pradesh	1203.035043	653.8	1747.1	220.570812
East Rajasthan	656.501709	273.6	1350.4	166.163458
East Uttar Pradesh	975.864957	493.3	1545.5	203.806517

In [25]:

```
print("Case 2 - agg() on multiple columns at once:")
agg2 = df.groupby('SUBDIVISION').agg({'ANNUAL':'mean', 'JJAS':'mean',
'MONSOON_SHARE_PCT':'mean'})
agg2.head(10)
```

```
Case 2 - agg() on multiple columns at once:
```

Out[25]:

	ANNUAL	JJAS	MONSOON_SHARE_PCT
SUBDIVISION			
Andaman & Nicobar Islands	2926.373585	1703.708257	58.411708
Arunachal Pradesh	3403.959140	2258.214433	65.598932
Assam & Meghalaya	2579.133333	1717.388889	66.670513
Bihar	1196.563248	1014.923932	84.851062
Chhattisgarh	1369.136752	1202.876923	88.077790
Coastal Andhra Pradesh	1050.304274	655.380342	63.049750
Coastal Karnataka	3397.637931	2973.528205	87.482073
East Madhya Pradesh	1203.035043	1075.136752	89.488544
East Rajasthan	656.501709	604.490598	91.966219
East Uttar Pradesh	975.864957	859.070085	87.984516

14. Selection on particular groups (by name / condition)¶

In [26]:

```
grouped = df.groupby('SUBDIVISION')
print("get_group('Konkan & Goa') - head:")
grouped.get_group('Konkan & Goa')[['YEAR','ANNUAL']].head()
```

```
get_group('Konkan & Goa') - head:
```

Out[26]:

	YEAR	ANNUAL
2551	1901	2792.9
2552	1902	2673.9
2553	1903	3084.3
2554	1904	2393.7

	YEAR	ANNUAL
2555	1905	1682.8

In [27]:

```
print("Subdivisions whose average ANNUAL rainfall > 3000mm:")
high_rain = g1[g1 > 3000]
high_rain
```

```
Subdivisions whose average ANNUAL rainfall > 3000mm:
```

Out[27]:

```
SUBDIVISION
Arunachal Pradesh    3403.959140
Coastal Karnataka    3397.637931
Name: ANNUAL, dtype: float64
```

15. Correlation between two columns¶

In [28]:

```
corr_val = df['JJAS'].corr(df['ANNUAL'])
print(f"Correlation between JJAS (monsoon) rainfall and ANNUAL rainfall:
{corr_val:.4f}")
```

```
Correlation between JJAS (monsoon) rainfall and ANNUAL rainfall: 0.9430
```

In [29]:

```
print("Full correlation matrix of monthly + seasonal columns:")
df[month_cols + season_cols].corr().round(2)
```

```
Full correlation matrix of monthly + seasonal columns:
```

Out[29]:

	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	ANNUAL	JF	MAM	JJ
JAN	1.00	0.45	0.39	0.21	0.13	-0.03	-0.05	0.01	0.02	0.01	0.07	0.23	0.11	0.84	0.24	-0
FEB	0.45	1.00	0.58	0.37	0.21	0.04	0.02	0.07	0.08	-0.00	-0.02	0.13	0.19	0.86	0.39	0.
MAR	0.39	0.58	1.00	0.56	0.37	0.17	0.10	0.14	0.18	0.09	0.01	0.13	0.33	0.57	0.65	0.
APR	0.21	0.37	0.56	1.00	0.65	0.45	0.27	0.26	0.38	0.37	0.16	0.13	0.58	0.34	0.87	0.
MAY	0.13	0.21	0.37	0.65	1.00	0.57	0.33	0.33	0.49	0.53	0.36	0.25	0.70	0.20	0.91	0.
JUN	-0.03	0.04	0.17	0.45	0.57	1.00	0.74	0.65	0.55	0.49	0.23	0.09	0.89	0.00	0.54	0.

	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	ANNUAL	JF	MAM	JJ
JUL	-0.05	0.02	0.10	0.27	0.33	0.74	1.00	0.69	0.51	0.30	0.04	-0.02	0.81	-0.02	0.32	0.
AUG	0.01	0.07	0.14	0.26	0.33	0.65	0.69	1.00	0.50	0.25	0.02	0.00	0.76	0.05	0.32	0.
SEP	0.02	0.08	0.18	0.38	0.49	0.55	0.51	0.50	1.00	0.39	0.16	0.12	0.72	0.06	0.47	0.
OCT	0.01	-0.00	0.09	0.37	0.53	0.49	0.30	0.25	0.39	1.00	0.48	0.29	0.59	0.01	0.47	0.
NOV	0.07	-0.02	0.01	0.16	0.36	0.23	0.04	0.02	0.16	0.48	1.00	0.46	0.31	0.03	0.27	0.
DEC	0.23	0.13	0.13	0.13	0.25	0.09	-0.02	0.00	0.12	0.29	0.46	1.00	0.21	0.21	0.23	0.
ANNUAL	0.11	0.19	0.33	0.58	0.70	0.89	0.81	0.76	0.72	0.59	0.31	0.21	1.00	0.17	0.70	0.
JF	0.84	0.86	0.57	0.34	0.20	0.00	-0.02	0.05	0.06	0.01	0.03	0.21	0.17	1.00	0.37	0.
MAM	0.24	0.39	0.65	0.87	0.91	0.54	0.32	0.32	0.47	0.47	0.27	0.23	0.70	0.37	1.00	0.
JJAS	-0.02	0.05	0.17	0.39	0.50	0.89	0.91	0.84	0.70	0.42	0.13	0.05	0.94	0.02	0.47	1.
OND	0.10	0.02	0.09	0.32	0.53	0.41	0.19	0.16	0.33	0.86	0.81	0.61	0.53	0.07	0.45	0.

16. Transformation - normalization (Min-Max scaling)¶

In [30]:

```
df['ANNUAL_NORM'] = (df['ANNUAL'] - df['ANNUAL'].min()) / (df['ANNUAL'].max() -
df['ANNUAL'].min())
df[['SUBDIVISION','YEAR','ANNUAL','ANNUAL_NORM']].head()
```

Out[30]:

	SUBDIVISION	YEAR	ANNUAL	ANNUAL_NORM
0	Andaman & Nicobar Islands	1901	3373.2	0.528155
1	Andaman & Nicobar Islands	1902	3520.7	0.551685
2	Andaman & Nicobar Islands	1903	2957.4	0.461827
3	Andaman & Nicobar Islands	1904	3079.6	0.481320
4	Andaman & Nicobar Islands	1905	2566.7	0.399502

In [31]:

```
print("Range check after normalization -> min:", df['ANNUAL_NORM'].min(), " max:",
df['ANNUAL_NORM'].max())
```

```
Range check after normalization -> min: 0.0  max: 1.0
```

17. Joining, merging, concatenation¶

In [32]:

```
# small lookup table for a merge demo
state_lookup = pd.DataFrame({
    'SUBDIVISION': ['Konkan & Goa', 'Kerala', 'Coastal Karnataka', 'Gangetic West
Bengal'],
    'REGION_TYPE': ['Coastal', 'Coastal', 'Coastal', 'Gangetic Plain']
})
merged = pd.merge(df, state_lookup, on='SUBDIVISION', how='left')
print("Merged dataframe (only matched rows shown):")
merged[merged['REGION_TYPE'].notnull()][['SUBDIVISION','YEAR','REGION_TYPE']].head()
```

```
Merged dataframe (only matched rows shown):
```

Out[32]:

	SUBDIVISION	YEAR	REGION_TYPE
562	Gangetic West Bengal	1901	Gangetic Plain
563	Gangetic West Bengal	1902	Gangetic Plain
564	Gangetic West Bengal	1903	Gangetic Plain
565	Gangetic West Bengal	1904	Gangetic Plain
566	Gangetic West Bengal	1905	Gangetic Plain

In [33]:

```
print("Concat demo (first 3 + last 3 rows):")
concat_demo = pd.concat([df.head(3), df.tail(3)])
concat_demo[['SUBDIVISION','YEAR','ANNUAL']]
```

```
Concat demo (first 3 + last 3 rows):
```

Out[33]:

	SUBDIVISION	YEAR	ANNUAL
0	Andaman & Nicobar Islands	1901	3373.2
1	Andaman & Nicobar Islands	1902	3520.7

	SUBDIVISION	YEAR	ANNUAL
2	Andaman & Nicobar Islands	1903	2957.4
4185	Lakshadweep	2015	1642.9
4186	Lakshadweep	2016	1065.7
4187	Lakshadweep	2017	1738.9

Conclusion¶

All core Pandas/NumPy data-handling operations were carried out on the real IMD subdivision-wise rainfall dataset (4188 records, 36 subdivisions, 1901-2017): reading, exploration, null detection + imputation, sorting/filtering, frequency counts, loc/iloc indexing, compound-condition row access, min/max analysis, groupby + aggregation, a derived column, Min-Max normalization, and merging/concatenation. The strongest finding: JJAS (monsoon) rainfall correlates 0.94 with ANNUAL rainfall, meaning monsoon-season data alone drives most of a subdivision's yearly total - directly useful for the KJS-AGR-01 irrigation advisory model.